

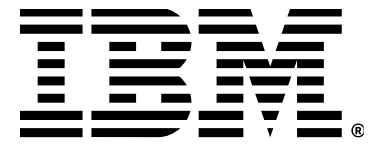
AssetOpsBench: Benchmarking AI Agents for Task Automation in Industrial Asset Operation and Maintenance

Dr. Dhaval Patel
Eng. Shuxin Lin
Dr. Nianjun Zhou
Dr. James Rayfield

IBM

<https://github.com/IBM/AssetOpsBench>





Agenda:

- Enterprise Benchmark
- Industrial AI
- Agents
- Models
- Open-Source Works



Rise of Enterprise Benchmark

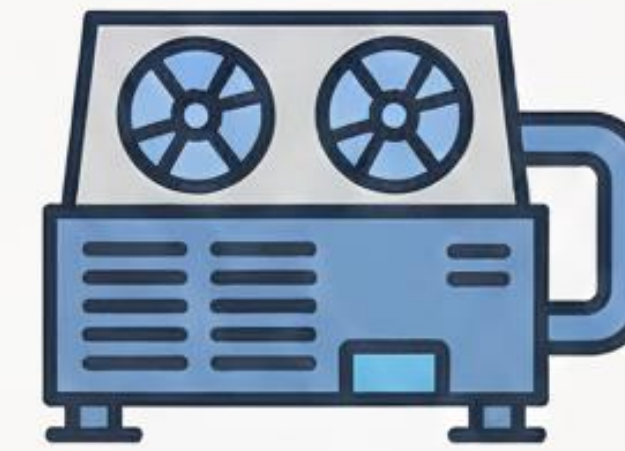
<https://www.kaggle.com/benchmarks>

[illegible]

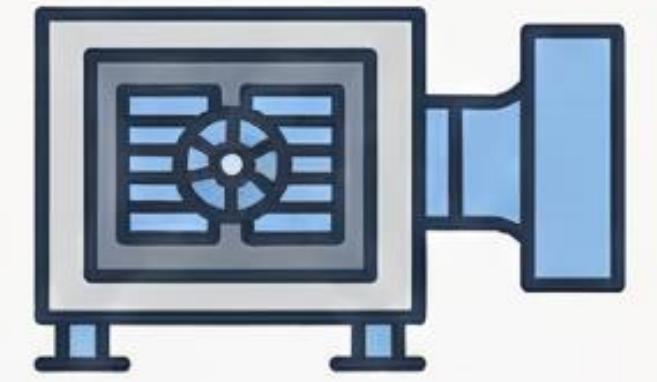
Industrial Assets

We are all surrounded by industrial **assets** - often without noticing them. These systems form the backbone of critical infrastructure and everyday services:

- **Data Centers:** Chillers, air-handling units, standby generators
- **Hospitals:** Standby generators ensuring uninterrupted operations
- **Energy Generation:** Wind turbines powering renewable electricity
-



Chiller



Air-Handling Unit



Standby Generator



Wind Turbine



Transportation

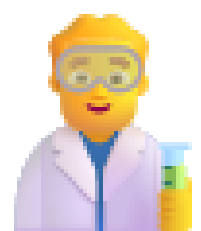


Air Production Unit

Monitoring and Maintenance Tasks

With the emergence of Industry 4.0 applications, these assets are now being monitored using 100s of sensors in real time:

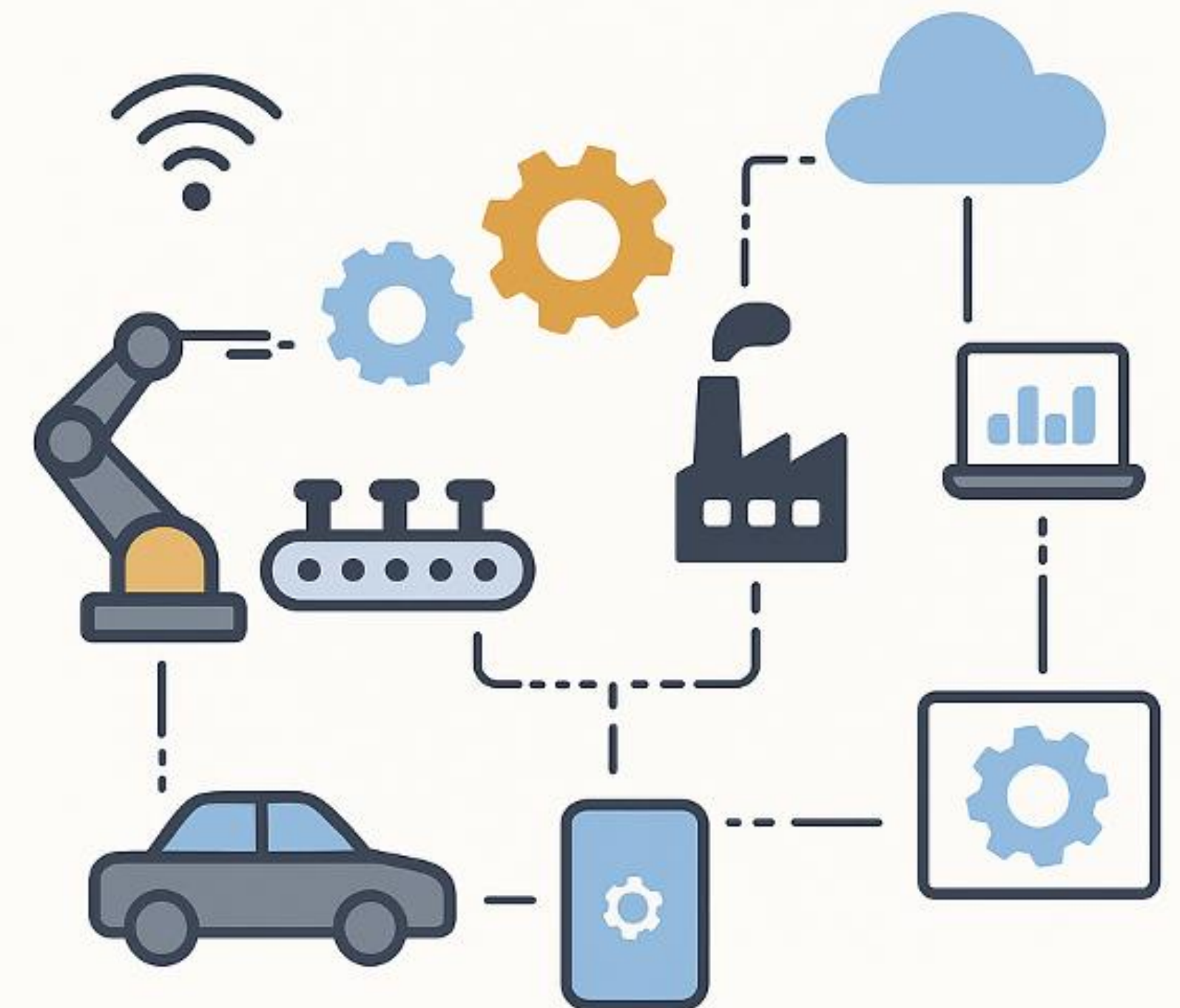
- 🌡️ Temperature,
- 🔵 Pressure,
- 🌀 Vibration,
- ⚡ Power,
- 💧 Flow,
- 🔊 Acoustic,
- 🌬️ Airflow,
- 📊 RPM,
- 🔥 Thermal,
- 🧪 Gas,
- 🔍 Visual



Is Chiller 11's compressor overheating? Generate a service request if needed.

Industry 4.0

The integration of physical machines with digital technologies like IoT, AI, cloud computing, robotics, and digital twins to create intelligent, automated, and interconnected industrial systems



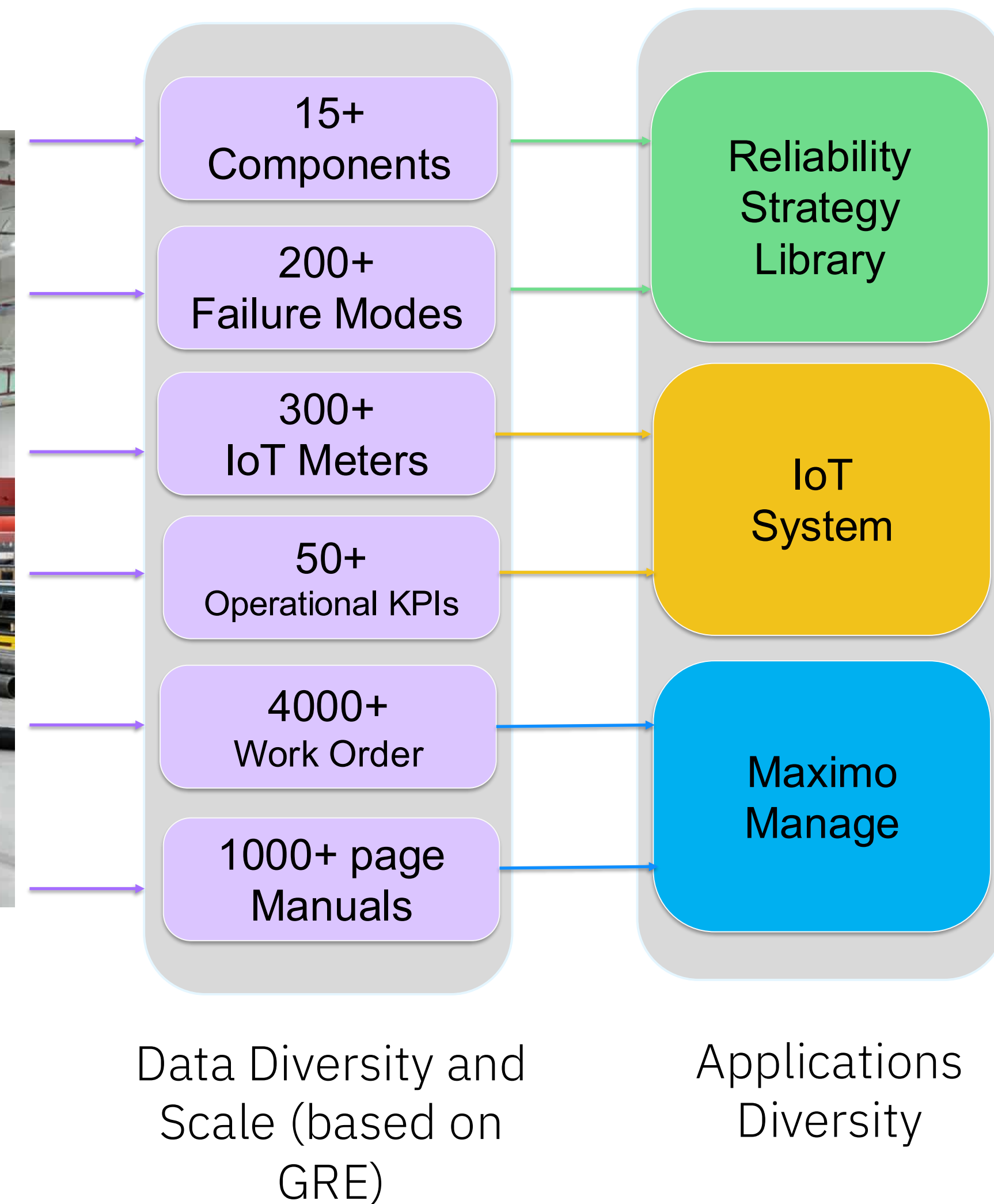
AI Agents for Industrial Assets : Example

Industrial assets are complex systems that generate a wealth of data through the monitoring of their lifecycle (ALM)

E.g., Data Center Chiller



This complexity leads to dynamics workflows depending on the alarm/condition of the chiller

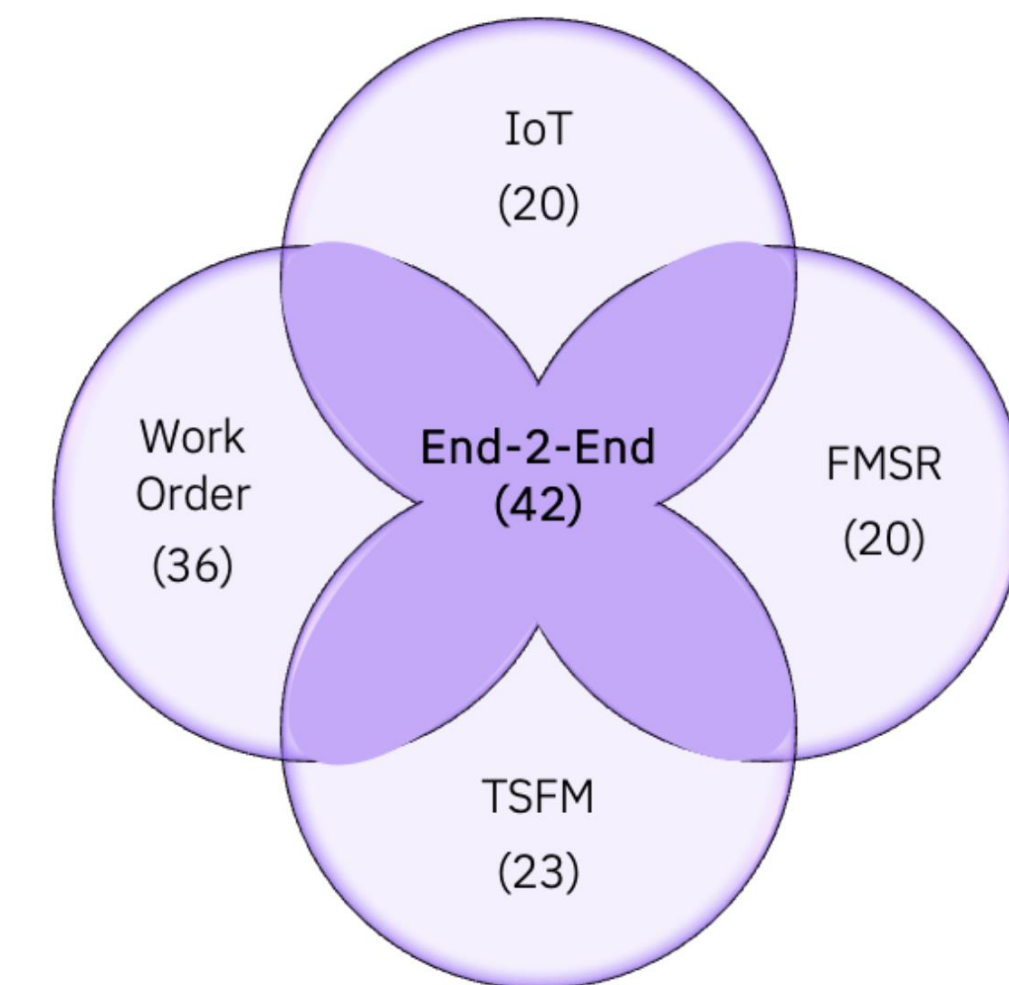
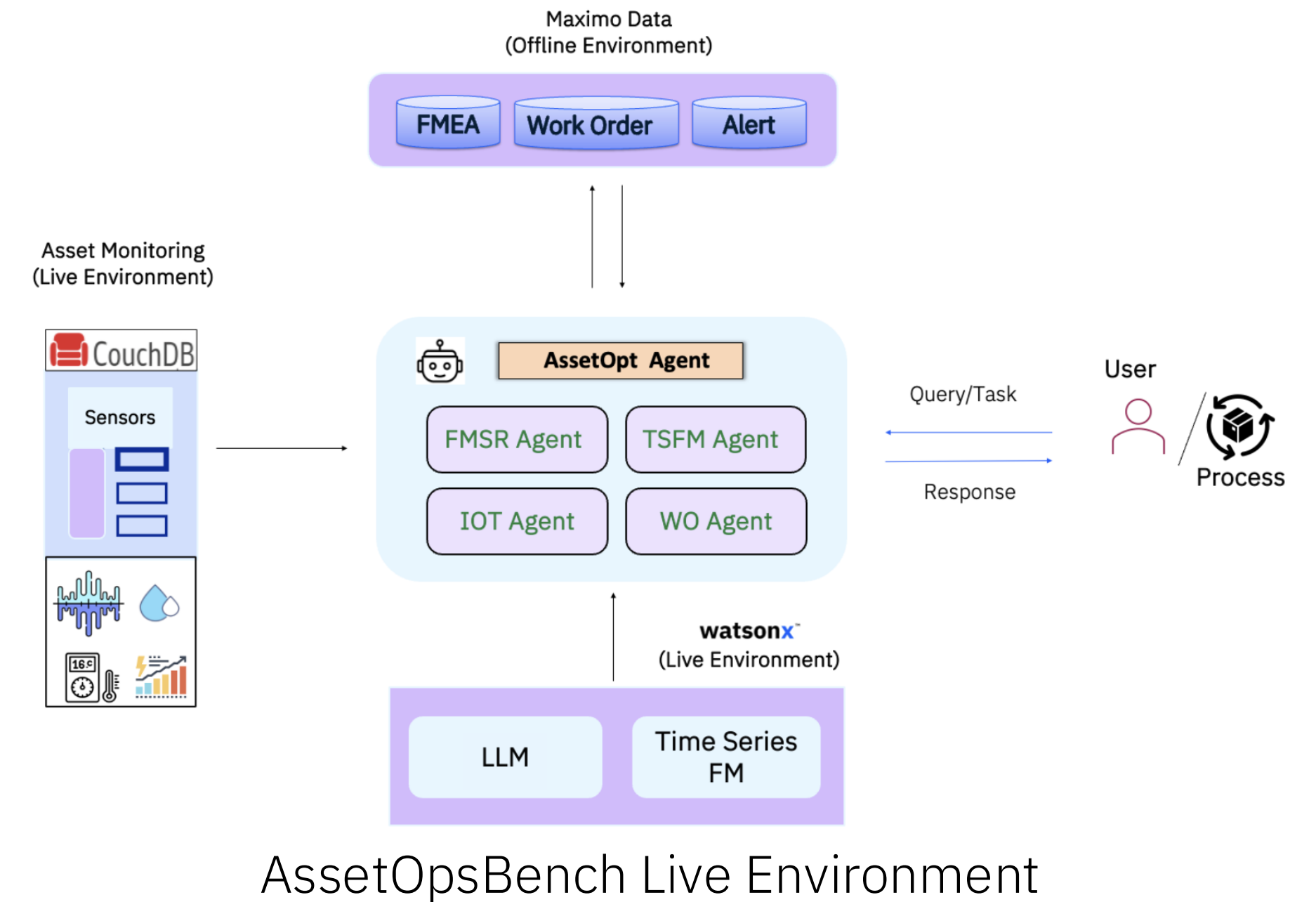


AssetOpsBench : Open-Source Initiative



AssetOpsBench: Open-Source Benchmark for Industry 4.0

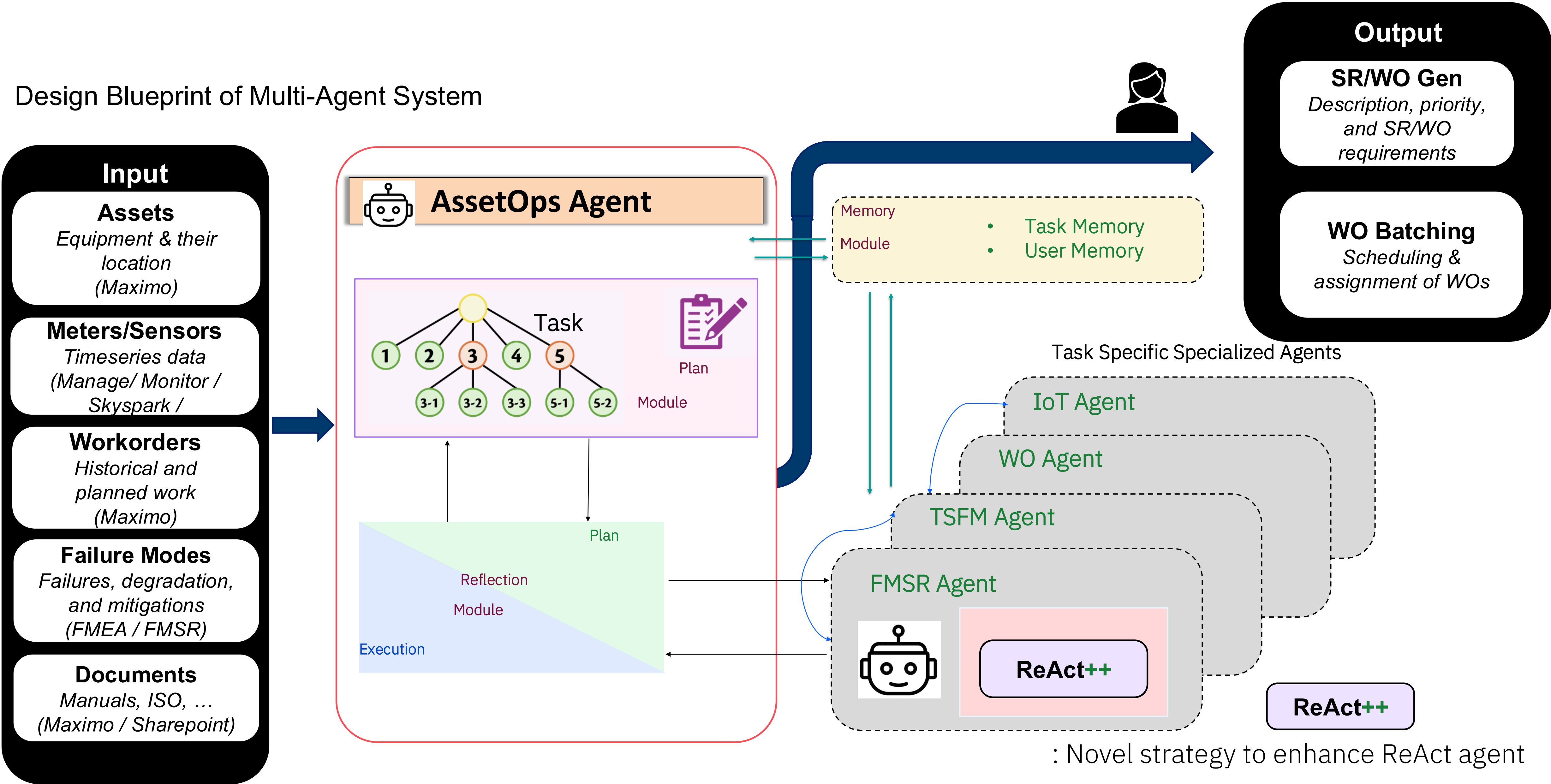
- Framework to assess Gen AI solutions' ability to solve Industry 4.0 Automation "**Scenarios**"
- **Simulated** industrial environment, **9 multi-source data sets** (work orders, FMEAs, timeseries) and **4 domain-specific agents** (IoT, data science, work order, failure mode to sensor mapping)
- **140+** expert-authored natural language queries, grounded in **enterprise industrial scenarios**
- Two Multi-Agent Orchestration Recipes
 - **Agent-As-Tool**
 - **Plan-Execute**
- **LLM-as-Judge** for Rubric-based Agent Evaluation and **Reference-based** Scoring for Semantic Evaluation
- **Agent harness**: systematic procedure for automated discovery of emerging failure modes



141 Utterance Distribution

Architecture: Agentic Asset Management

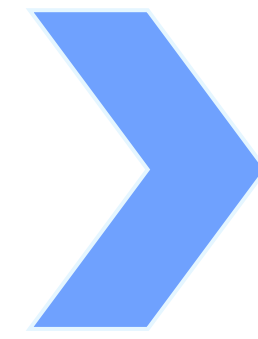
Design Blueprint of Multi-Agent System



An Example of Agentic Workflow Patterns in Industrial Setting

Inputs Data:

- FMEAs
- Asset Data
- IoT Meters
- WO History



Agentic Workflow



FMSR Agent

*Failure Mode to
Sensor Relationship
Mapping*

IoT Agent

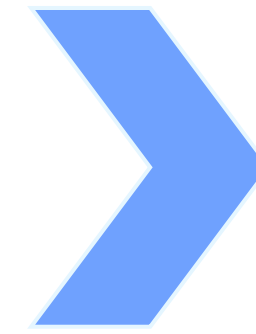
*Access to IoT and
other Time Series
Data*

TSFM Agent

*Automate Time Series
Analysis using
pretrained TTM, etc.*

Work Order (WO) Agent

*Automate Workorder
Analysis & Generation*



Outputs:

- WO Recommendation
- Smart Alerts
- Maintenance Insights

User Query

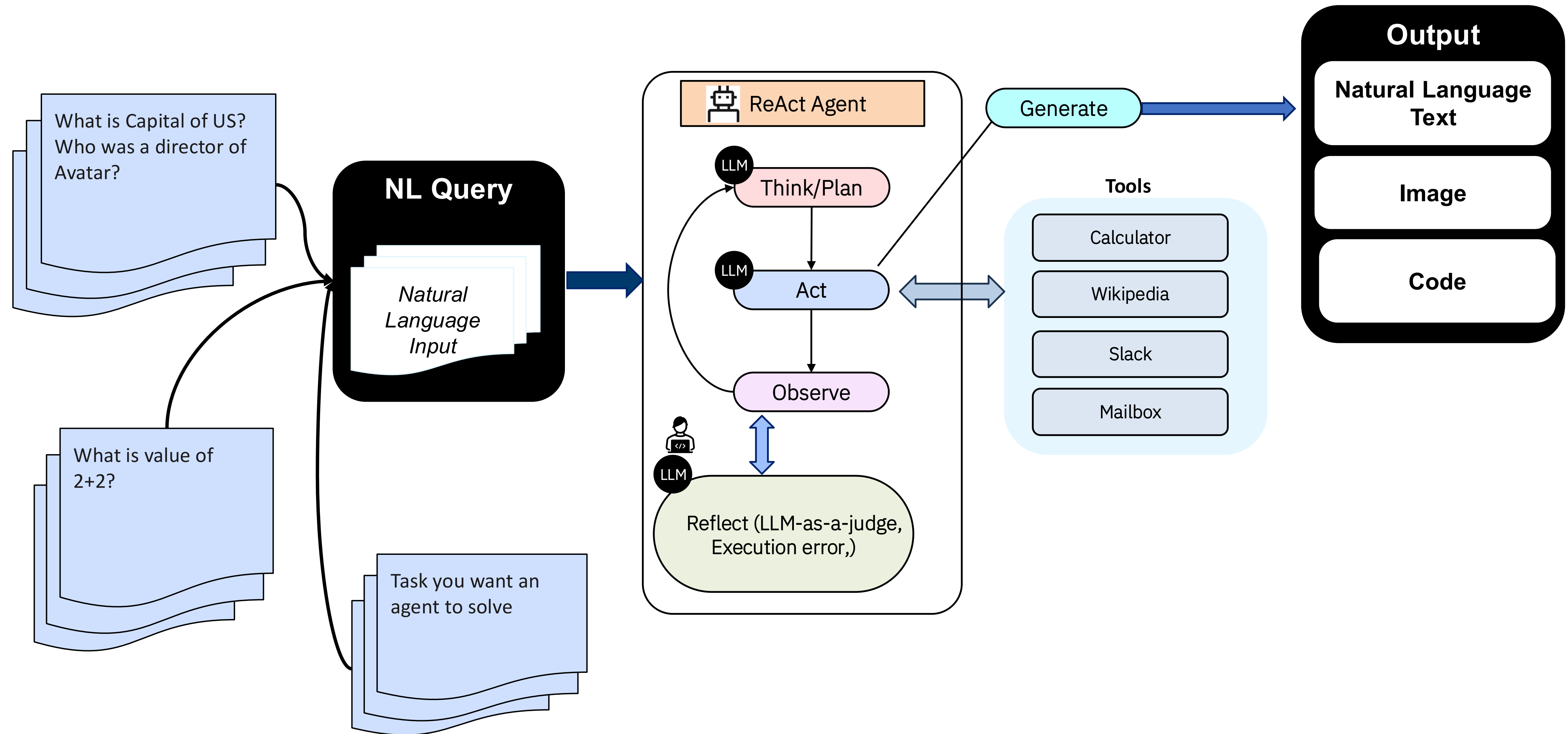
Example of Tasks → two broad categories of queries:

- **Knowledge Query** such as “*was there a performance issue over the last month in chiller 9*”?
- **Decision Support Query** such as – “*recommend a work order based on last three month’s health condition of Chiller 9*”? (health condition)

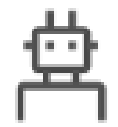
Challenges for AI Agent

- Integrate diverse data sources
- Automatically identify key sensors and KPIs.
- Adaptively choose the optimal model for anomaly detection.
- Monitor asset health using historical alerts and anomalies
- Generate actionable maintenance recommendations.

Architecture: Vanilla Agent



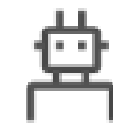
Task Specific Specialized Agents



IOT Agent

Q_10_trajectory_output.json

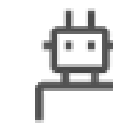
Get sensor data for both Chiller 6 and Chiller 9's Tonnage from last week at POKMAIN.



FMSR Agent

Q_14_trajectory_output.json

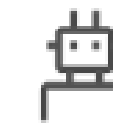
If compressor overheating occurs for Chiller 6, which sensor should be prioritized for monitoring this specific failure?



TSFM Agent

Q_22_trajectory_output.json

Explore and use several relevant methods to analyze time series data sensor 'Chiller 9 Condenser Water Flow' using data in...



WO Agent

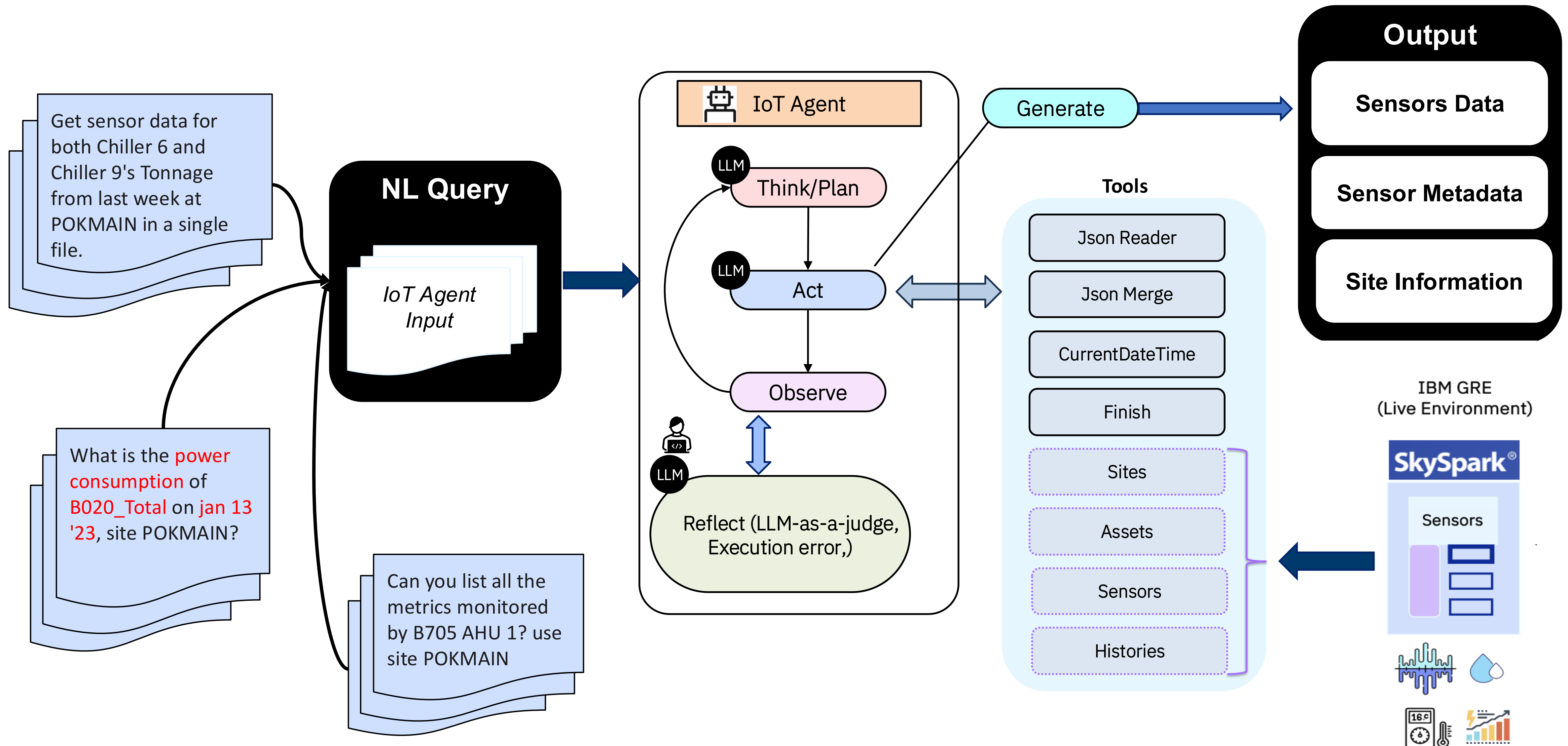
Q_31_trajectory_output.json

Assume today is early of July 2020, I would like to review the performance of chiller 9 with equipment ID CU02009 for June 2020...



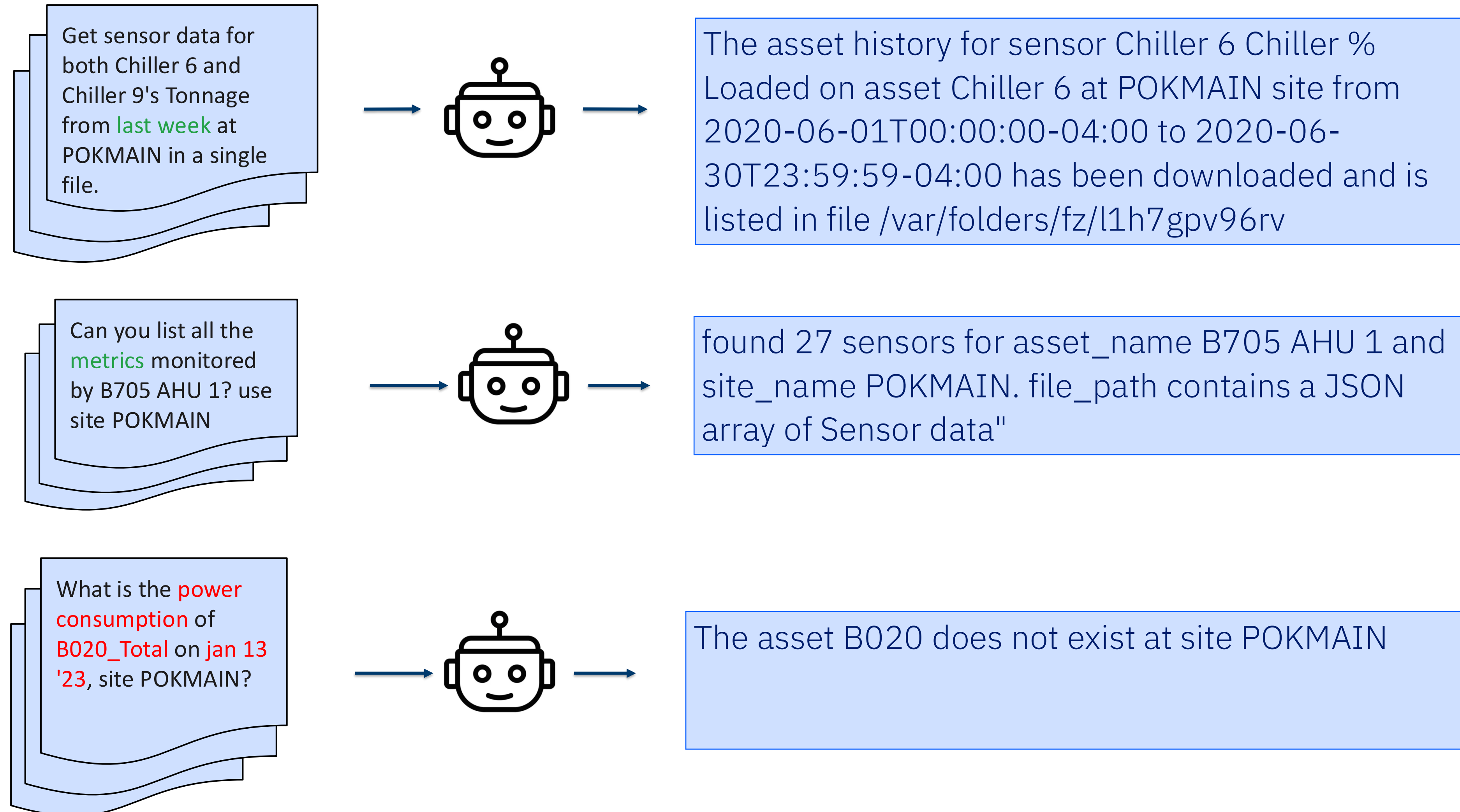
Architecture: IoT Agent

From NL to Data

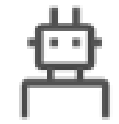


IoT Agent : **Input** and **Output**

From NL to Data



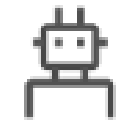
Task Specific Specialized Agents



IOT Agent

Q_10_trajectory_output.json

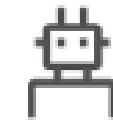
Get sensor data for both Chiller 6 and Chiller 9's Tonnage from last week at POKMAIN.



FMSR Agent

Q_14_trajectory_output.json

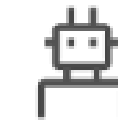
If compressor overheating occurs for Chiller 6, which sensor should be prioritized for monitoring this specific failure?



TSFM Agent

Q_22_trajectory_output.json

Explore and use several relevant methods to analyze time series data sensor 'Chiller 9 Condenser Water Flow' using data in...



WO Agent

Q_31_trajectory_output.json

Assume today is early of July 2020, I would like to review the performance of chiller 9 with equipment ID CU02009 for June 2020...



Architecture: FMSR (Failure Mode 2 Sensor Mapping) Agent

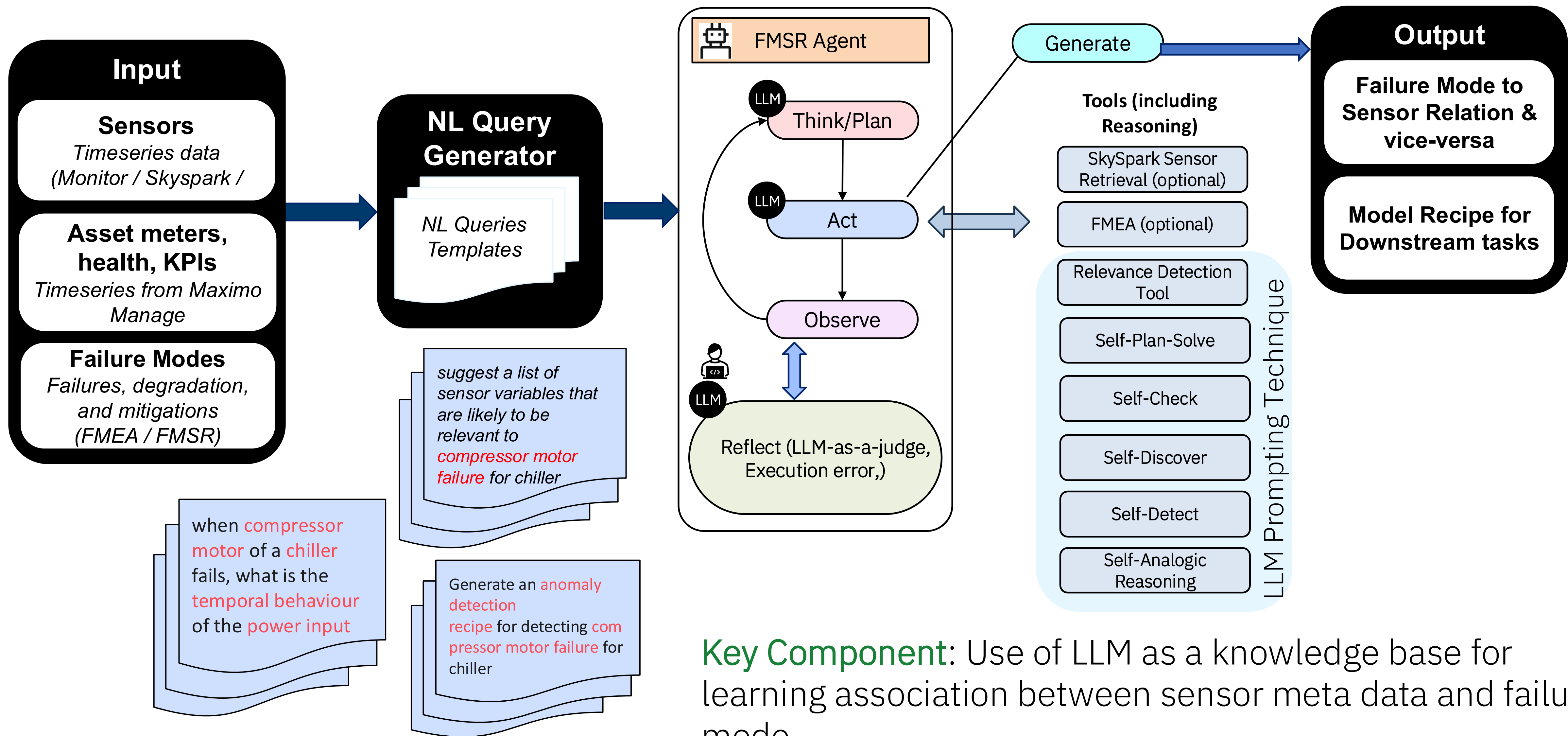
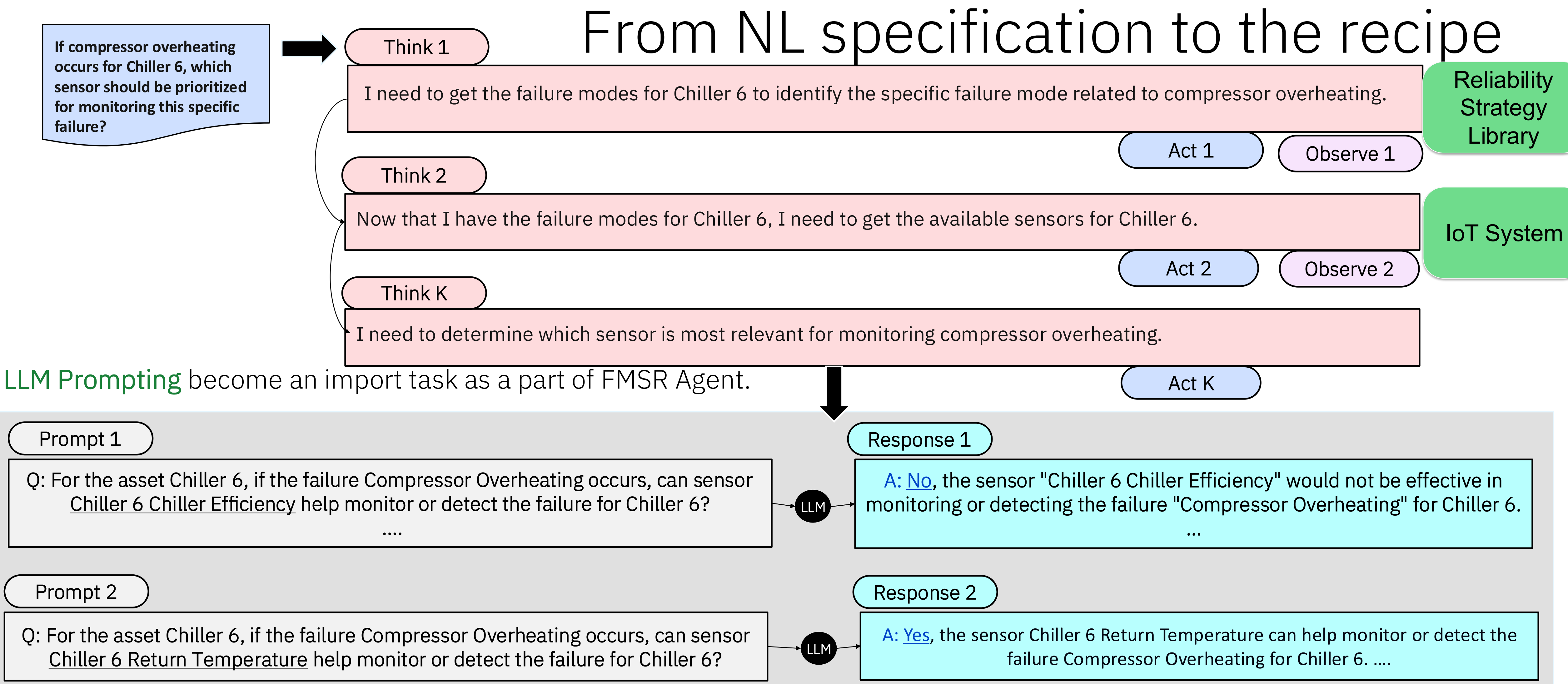
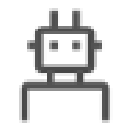


Illustration of FMSR Agent Working using Example



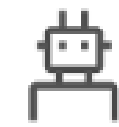
Task Specific Specialized Agents



IOT Agent

Q_10_trajectory_output.json

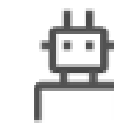
Get sensor data for both Chiller 6 and Chiller 9's Tonnage from last week at POKMAIN.



FMSR Agent

Q_14_trajectory_output.json

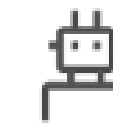
If compressor overheating occurs for Chiller 6, which sensor should be prioritized for monitoring this specific failure?



TSFM Agent

Q_22_trajectory_output.json

Explore and use several relevant methods to analyze time series data sensor 'Chiller 9 Condenser Water Flow' using data in...



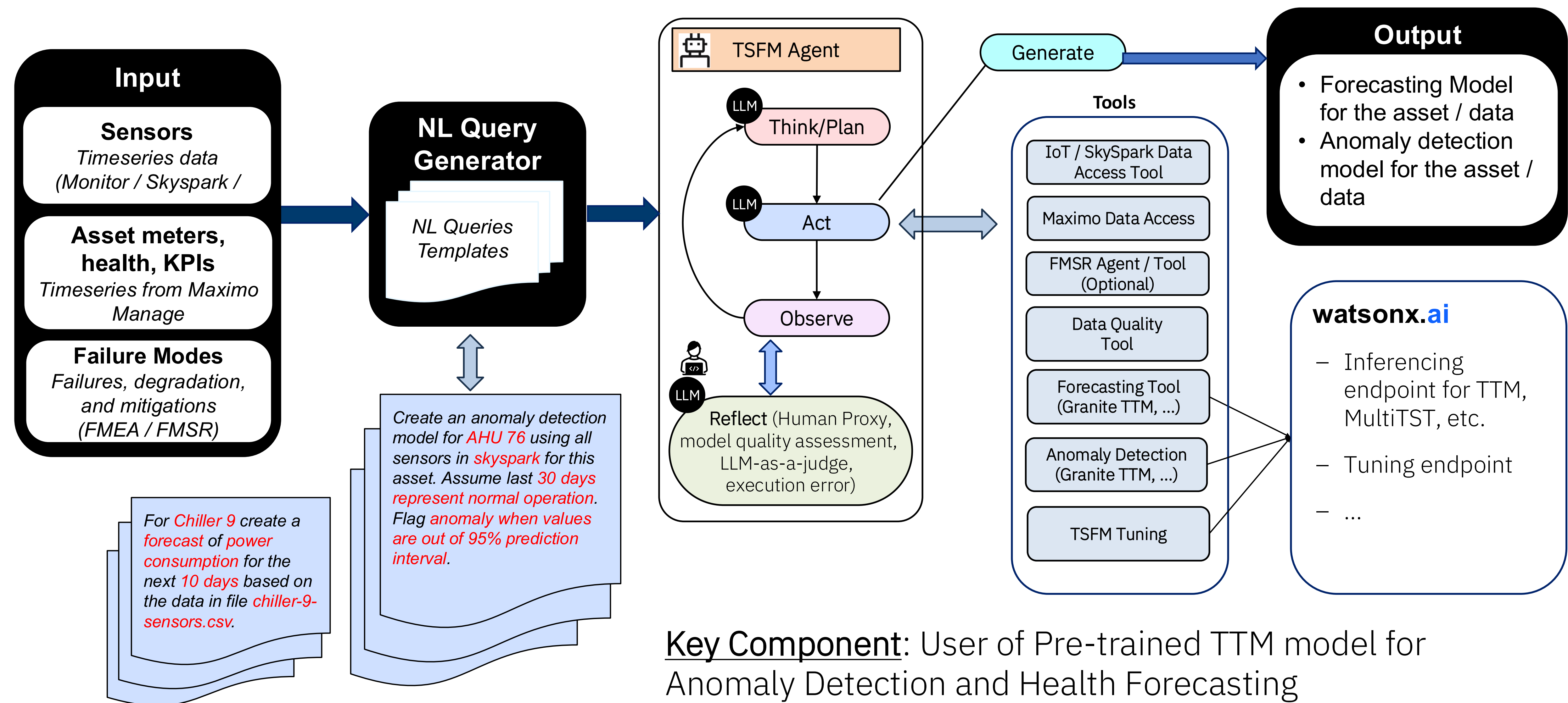
WO Agent

Q_31_trajectory_output.json

Assume today is early of July 2020, I would like to review the performance of chiller 9 with equipment ID CU02009 for June 2020...

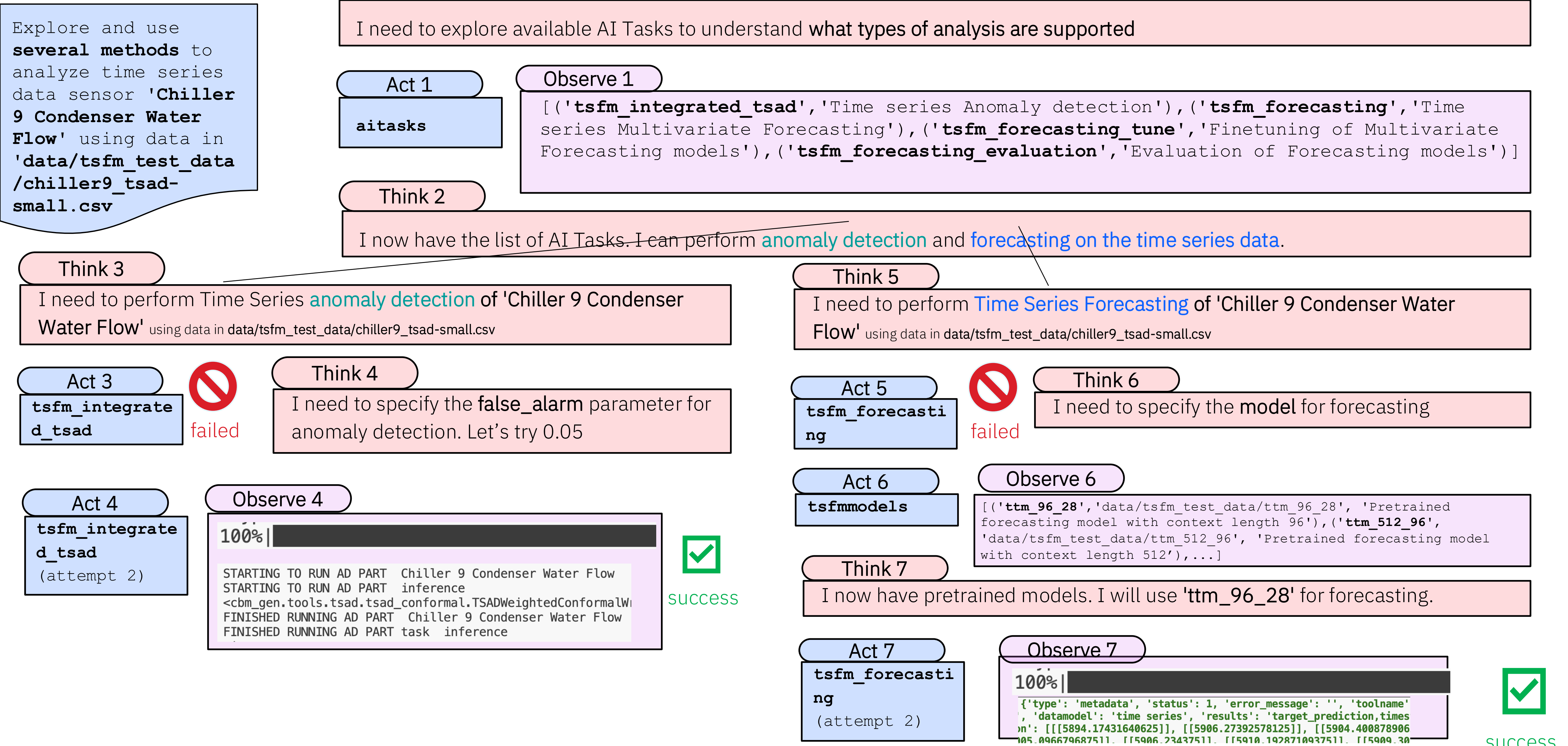


Architecture: TSFM (Time Series Foundation Model) Agent

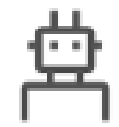


Architecture: TSFM (Time Series Foundation Model) Agent

Interesting Trajectory of TSFM Agent



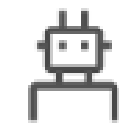
Task Specific Specialized Agents



IOT Agent

Q_10_trajectory_output.json

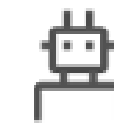
Get sensor data for both Chiller 6 and Chiller 9's Tonnage from last week at POKMAIN.



FMSR Agent

Q_14_trajectory_output.json

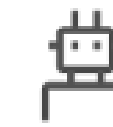
If compressor overheating occurs for Chiller 6, which sensor should be prioritized for monitoring this specific failure?



TSFM Agent

Q_22_trajectory_output.json

Explore and use several relevant methods to analyze time series data sensor 'Chiller 9 Condenser Water Flow' using data in...



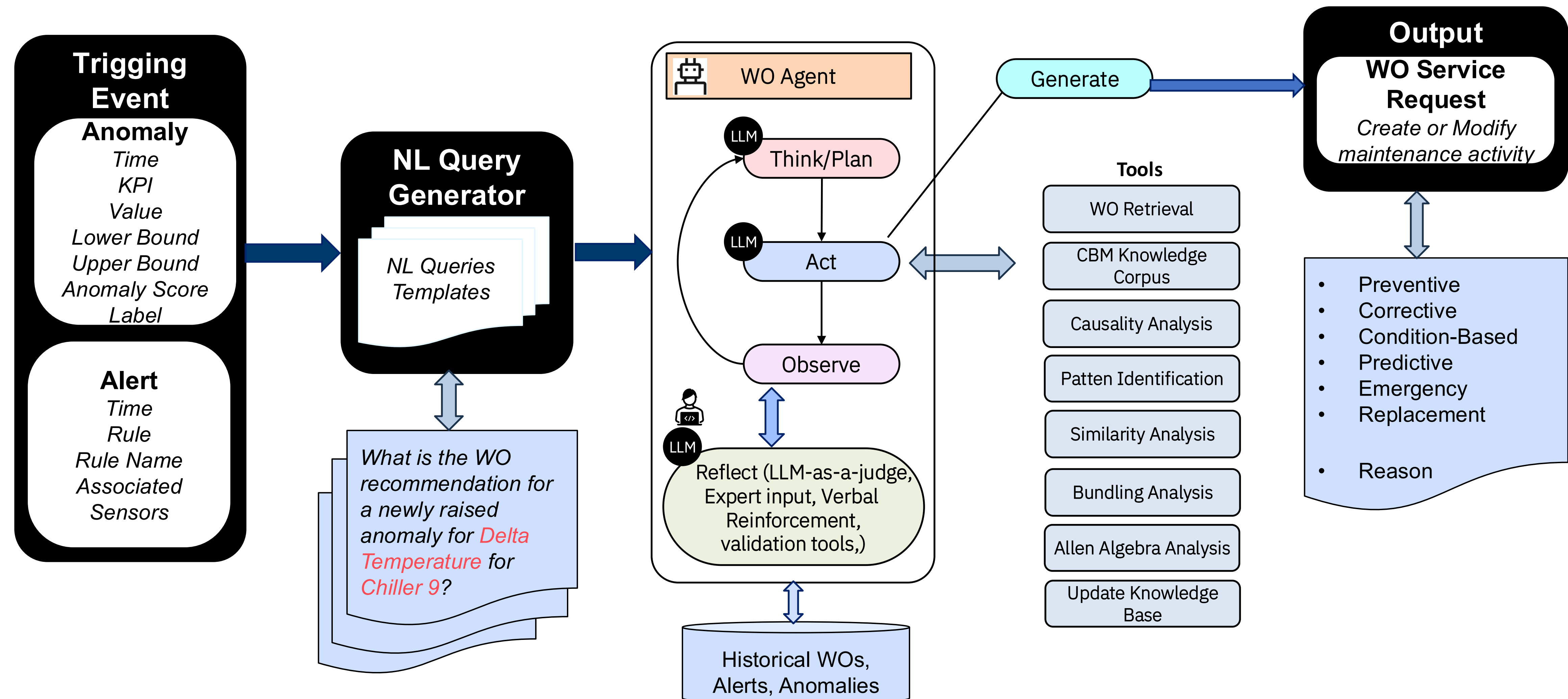
WO Agent

Q_31_trajectory_output.json

Assume today is early of July 2020, I would like to review the performance of chiller 9 with equipment ID CU02009 for June 2020...



Architecture: WO (Work Order) Agent



Key Component: LLM's ability to inspect structured output and code for doing temporal event analysis on Alert, Anomaly and Work-Orders

Interesting Trajectory of WO Agent

From NL specification to model

Monitor Chiller 9 and identify anomalous behaviors and provide a work order recommendation if any.

Think 1

To recommend a work order for Chiller 9 at the POKMAIN site, I need to first identify any anomalous behaviors within the specified date range. I will use the `get\_events` API to retrieve all events for Chiller 9 during this period and then filter for anomalies

Act 1

```
```python
Define the equipment and date range
equipment = Equipment(equipment_id="CU02009", site="POKMAIN")
date_range = DateRange(start_date=Date("2020-06-01"), end_date=Date("2020-06-01"))

Retrieve the events
events_pickle_path = get_events(equipment, date_range)

Load the work orders data from the pickle file
with open(work_orders_pickle_path, "rb") as file:
 work_orders = pickle.load(file)

Filter corrective work orders
corrective_work_orders = filter_corrective_work_orders(work_orders)

Display corrective work orders
for cwo in corrective_work_orders:
 print(cwo)```
```

### Observe 1

```
Event(event_id=a1_d460a5db-bfde-420c-9309-0a6d8bffb1b4, event_group=ALERT, event_category=ALERT, event_type=CR00008,
Event(event_id=a1_6c7a9e83-9434-4f2d-890f-87c056f0a2e8, event_group=ALERT, event_category=ALERT, event_type=CR00008,
Event(event_id=a1_d01ecec9-b076-4f36-85f6-77e7e5220e3f, event_group=ALERT, event_category=ALERT, event_type=CR00008,
Event(event_id=a1_c6deb4a0-b1ff-4a0d-acb6-12a19a5cc404, event_group=ALERT, event_category=ALERT, event_type=CR00008,
Event(event_id=an_c8c1176e-4464-4c85-bda9-5dd891494222, event_group=ANOMALY, event_category=ANOMALY, event_type=KPI
Event(event_id=an_e83c9bc0-2046-4a0c-a3f8-bc1c72f0d19b, event_group=ANOMALY, event_category=ANOMALY, event_type=KPI
Event(event_id=a1_6c86d0aa-4ee9-4c7d-8fbf-67063276fb9d, event_group=ALERT, event_category=ALERT, event_type=CR00008,
Event(event_id=a1_6b05fb2d-df7c-45f3-8a79-624a8e5ec1c2, event_group=ALERT, event_category=ALERT, event_type=CR00008,
```

# Interesting Trajectory of WO Agent

## From NL specification to model

Monitor Chiller 9 and identify anomalous behaviors and provide a work order recommendation if any.

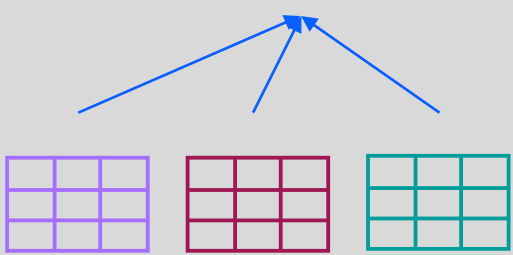
### Think 1

To recommend a work order for Chiller 9 at the POKMAIN site, I need to first identify any anomalous behaviors within the specified date range. I will use the `get\_events` API to retrieve all events for Chiller 9 during this period and then filter for anomalies .....

### Think 2

The data shows multiple anomalies and alerts related to the chiller's evaporator approach being high and various KPI anomalies. There is one corrective work order scheduled for a vibration analysis on June 16, 2020. Given the frequency and nature of the alerts and anomalies, it is likely that additional corrective work orders may be needed. To determine if new work orders should be generated, I will analyze the trend of these anomalies and alerts. Specifically, I will check if the frequency of these events is increasing over time.

Thanks to Data Classes and Structured input to LLM; thinking process improved



Data Classes

### Act 2

```
Count the number of anomalies and alerts per day
from collections import defaultdict

daily_event_count = defaultdict(int)
for event in anomaly_alert_events:
 event_date = event.event_time.strftime("%Y-%m-%d")
 daily_event_count[event_date] += 1

Display daily event counts
for date, count in sorted(daily_event_count.items()):
 print(f"{date}: {count} events")
```

### Observe 2

2020-06-01: 1 events  
2020-06-02: 1 events  
2020-06-03: 1 events  
2020-06-04: 1 events  
2020-06-05: 1 events  
2020-06-06: 1 events  
2020-06-08: 15 events  
2020-06-09: 5 events  
2020-06-10: 3 events

2020-06-16: 9 events  
2020-06-17: 1 events  
2020-06-18: 3 events  
2020-06-19: 2 events  
2020-06-21: 1 events  
2020-06-22: 2 events  
2020-06-23: 1 events  
2020-06-24: 1 events

# Interesting Trajectory of WO Agent

## From NL specification to model

**Monitor Chiller 9 and identify anomalous behaviors and provide a work order recommendation if any.**

### Think 1

To recommend a work order for Chiller 9 at the POKMAIN site, I need to first identify any anomalous behaviors within the specified date range. I will use the `get\_events` API to retrieve all events for Chiller 9 during this period and then filter for anomalies .....

### Think 2

The data shows multiple anomalies and alerts related to the chiller's evaporator approach being high and various KPI anomalies. There is one corrective work order scheduled for a vibration analysis on June 16, 2020. Given the frequency and nature of the alerts and anomalies, it is likely that additional corrective work orders may be needed. To determine if new work orders should be generated, I will analyze the trend of these anomalies and alerts. Specifically, I will check if the frequency of these events is increasing over time.

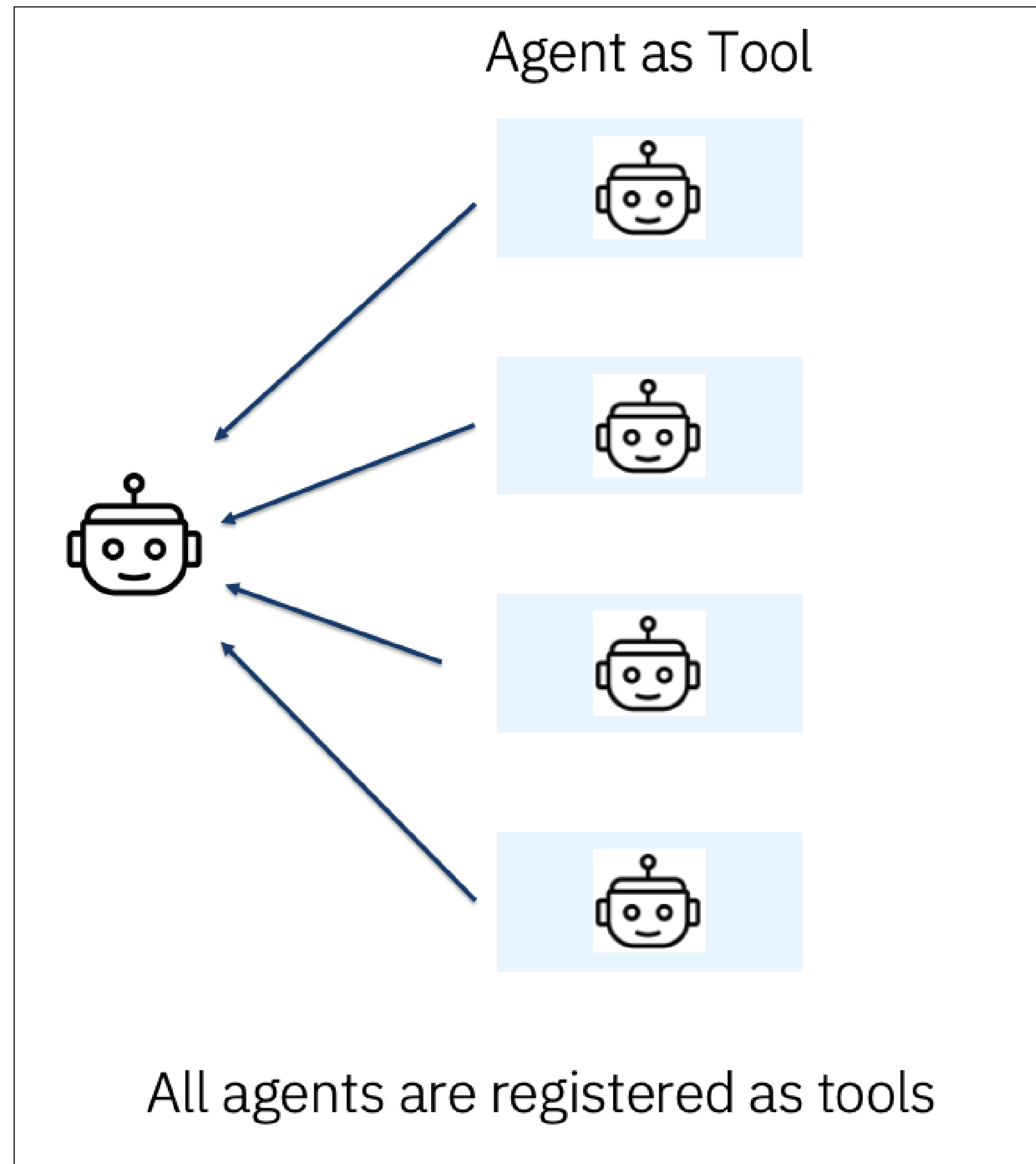
### Think 3

The daily event counts show a significant spike in anomalies and alerts on June 8 and June 16, with 15 and 9 events respectively. This indicates a potential issue that may require additional corrective work orders. To further analyze the situation, I will check if there are any pending work orders related to these issues. If not, I will recommend generating new work orders based on the nature of the anomalies and alerts.

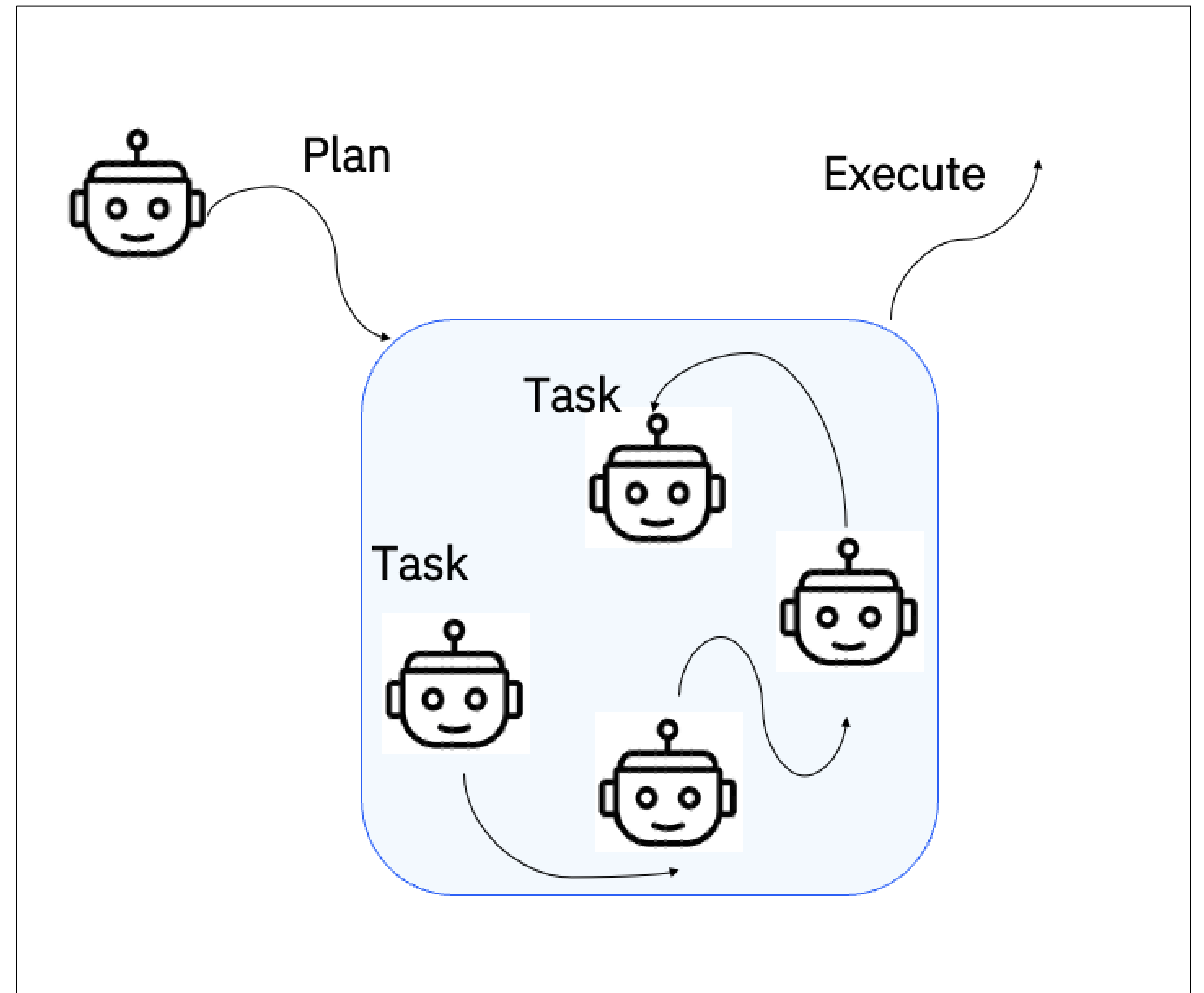
```
Final Answer:
{
 "corrective_work_orders": [
 {
 "wo_id": "L355477",
 "wo_description": "RESCAN CHILLER # 9",
 "collection": "entire chiller system",
 "primary_code": "MT013",
 "primary_code_description": "Vibration Analysis",
 "secondary_code": "MT013c",
 "secondary_code_description": "Post-Maintenance Vibration Analysis",
 "equipment_id": "CU02009",
 "equipment_name": "Chiller 9",
 "preventive": false,
 "work_priority": 1,
 "actual_finish": "2020-06-16 19:30:00",
 "duration": "0:00",
 "actual_work_hour": "0:00"
 }
],
 "recommendations": [
 {
 "recommendation": "Generate new work orders to address the chiller's"
 }
]
}
```



# AssetOpsBench : Multi-Agent Implementation Strategy



Agent-As-Tool Approach



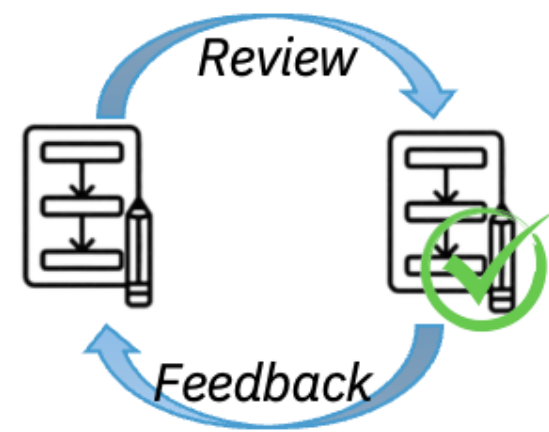
Plan-Execute Approach

# AssetOpsBench : Illustrative Workflow

## User Instruction

List all failure modes of Chiller 6 at MAIN site that can be detected by temperature sensors and power input sensors.

## Planning



### Step 1

Task: *List all sensors of Chiller 6 at MAIN site.*  
Agent: IoT  
Context: None

### Step 2

Task: *List all temperature sensors and power input sensors of Chiller 6 at MAIN site.*  
Agent: IoT  
Context: <Step1>

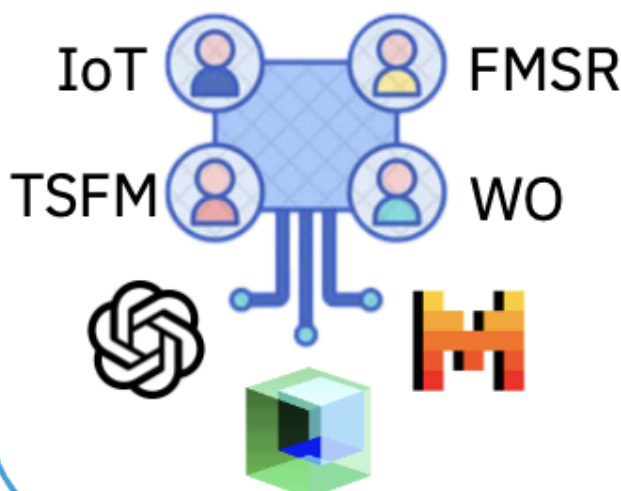
### Step 3

Task: *List all failure modes of asset Chiller 6 at MAIN site.*  
Agent: FMSR  
Context: None

### Step 4

Task: *List all failure modes of Chiller 6 at MAIN site that can be detected by temperature and power input sensors.*  
Agent: FMSR  
Context: <Step2>, <Step3>

## Execution



### Step 1

Response: *The sensors and metadata of Chiller 6 is downloaded and save in /tmp/...*

### Step 2

Response: *The temperature sensors are: ["Chiller 6 Condenser Water Return To Tower Temperature",...] and the power input sensor is: ["Chiller 6 Power Input"].*

### Step 3

Response: *"The failure modes of asset Chiller 6 are: 1. Compressor Overheating: Failed due to normal wear...*

### Step 4

Response: *The failure modes of Chiller 6 that can be detected by temperature sensors and power input sensors are: 1. Heat Exchangers: Fans: Degraded motor or worn bearing due to ...*

## Final Response

*The failure modes of Chiller 6 that can be detected by temperature sensors and power input sensors are: 1. Heat Exchangers: Fans: Degraded motor or worn bearing due to Normal use; 2. Evaporator Water side fouling; 3. Condenser Water side fouling.*

# AssetOpsBench : Evaluation





# Automatic Evaluation of Agentic Workflow

## Ground Truth Preparation

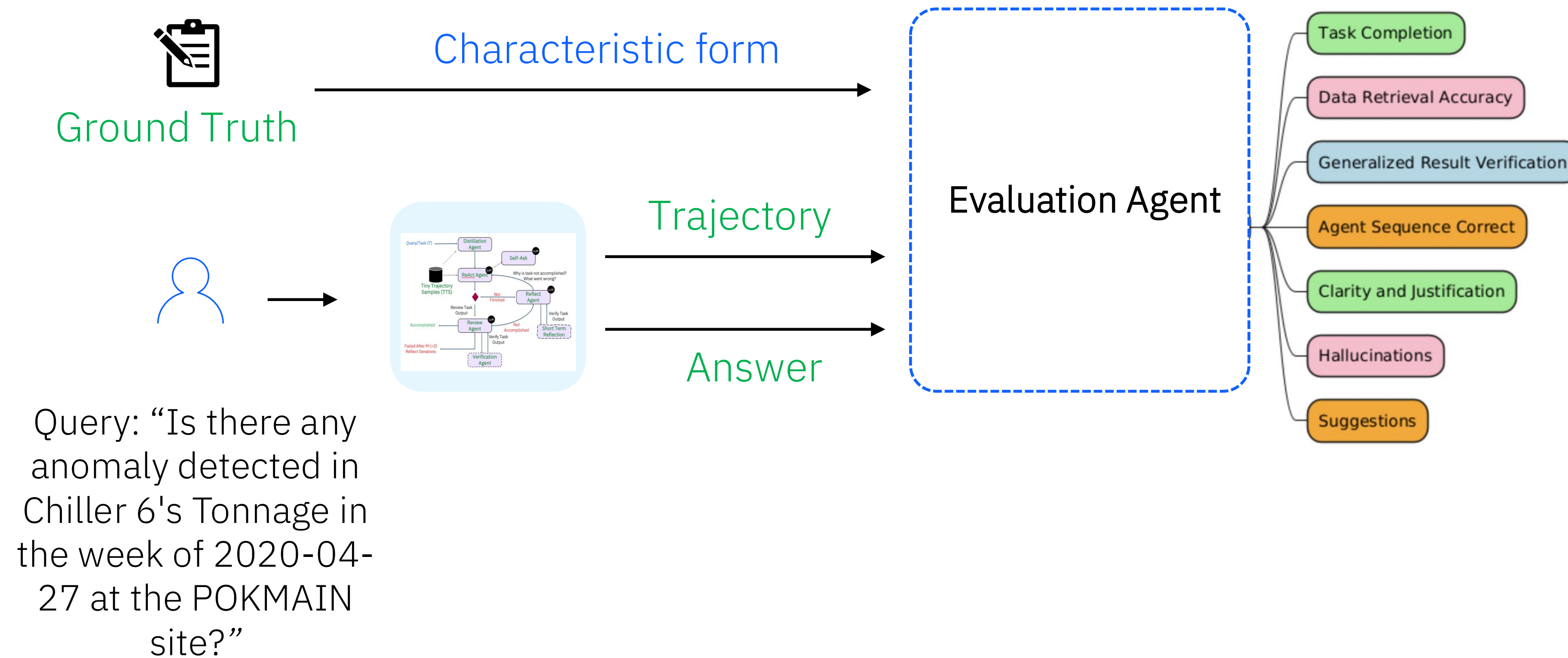
- **Characteristic form** describe the final output along with the process/step to be conducted. As an example:

“The expected response should confirm the successful execution of all required actions while ensuring the correct variables, including the asset (Chiller 6), location (POKMAIN), and time range (week of 2020-04-27), were used for data retrieval and analysis. It should specify that **IoTAgent** was called to request and download the data, and **TSFMAgent** was properly utilized to perform Time Series anomaly detection on the Tonnage parameter. The response must also verify that the data was accurately stored in the designated file location, and that the analysis results were saved to a new file. Additionally, the response should explicitly confirm the detection of anomalies in Chiller 6's Tonnage during the specified timeframe at the POKMAIN site, as these anomalies were anticipated.”

This characteristic form serves as the ground truth for evaluating responses using a **rubric-based performance assessment**.

# Automatic Evaluation of Agentic Workflow

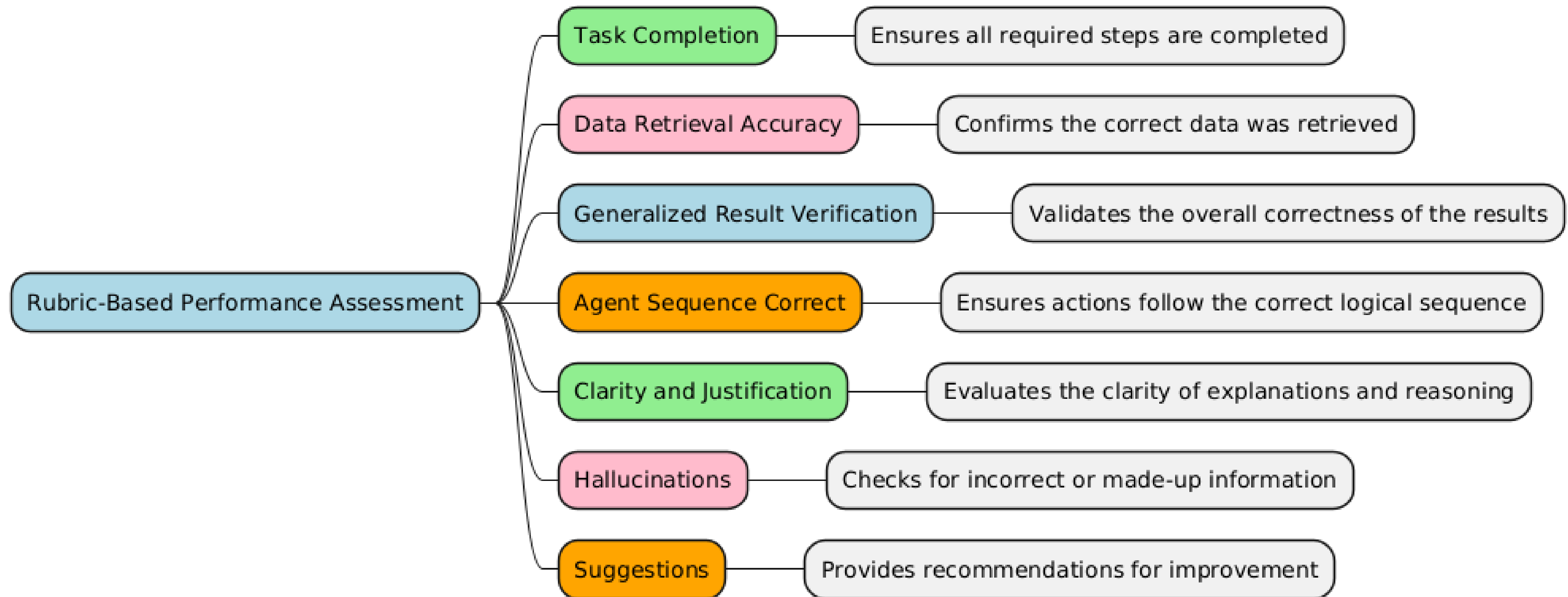
## Evaluation Agent: Rubric-based Performance Assessment



# Automatic Evaluation of Agentic Workflow

## Evaluation Agent: Rubric-based Performance Assessment

We captured performance across 6 metrics to better understand the failure mode of agents.





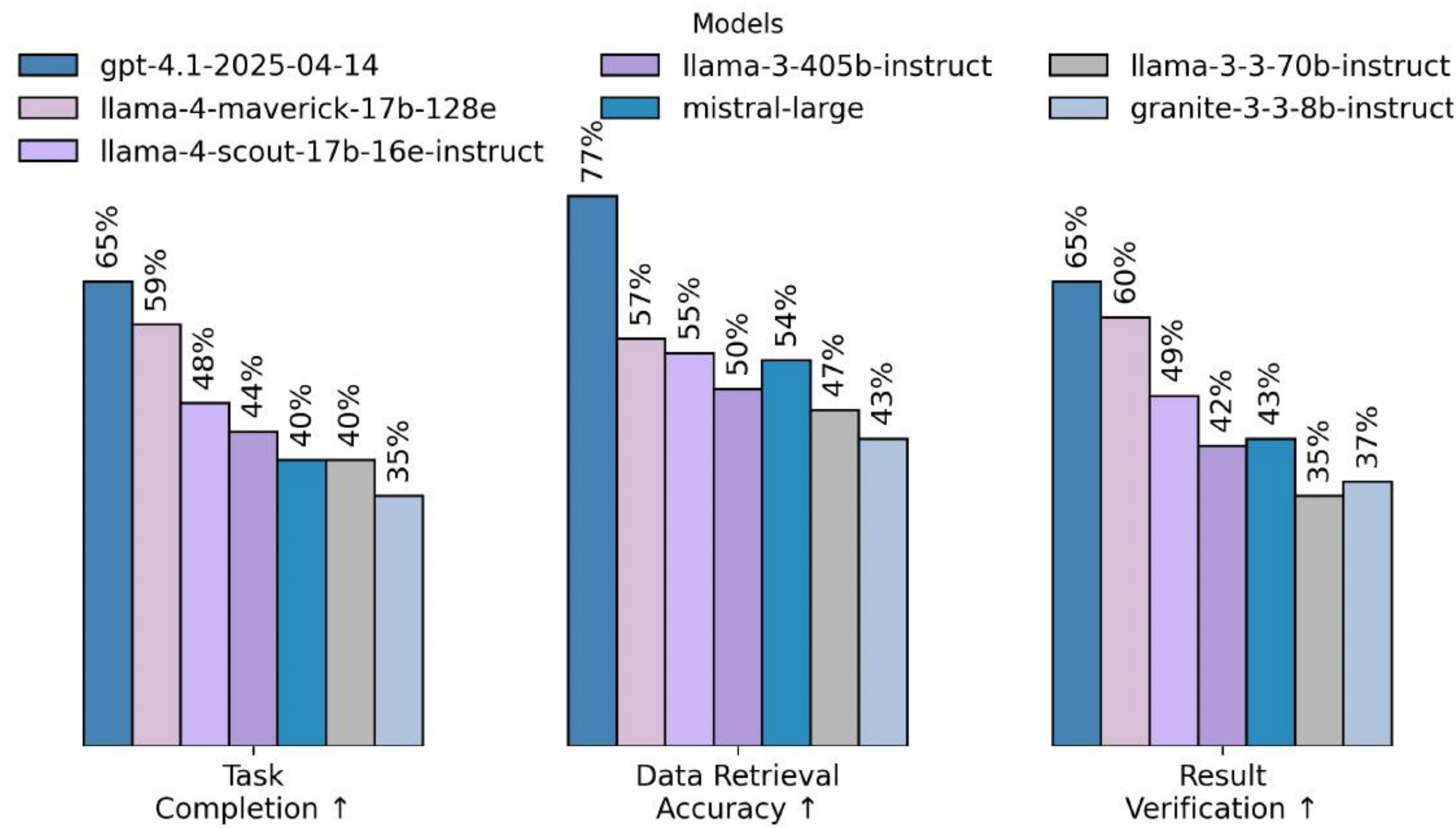
# AssetOpsBench : Summary of Result



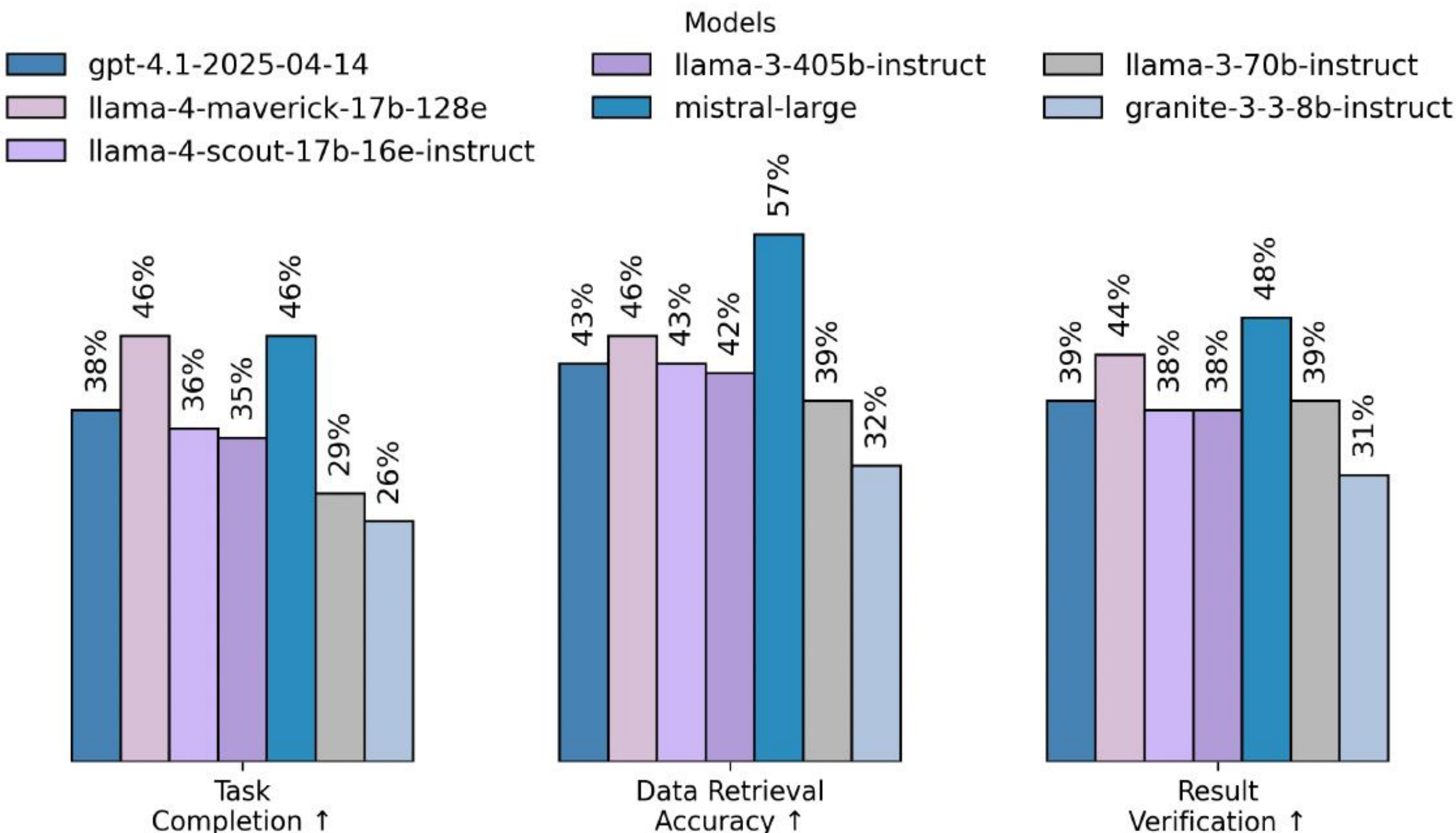
# Leaderboard

We conducted an extensive benchmark to compare the two orchestration recipes:

- Agent-As-Tool consistently outperforms Plan-Execute



((a)) **Agent-As-Tool** Approach



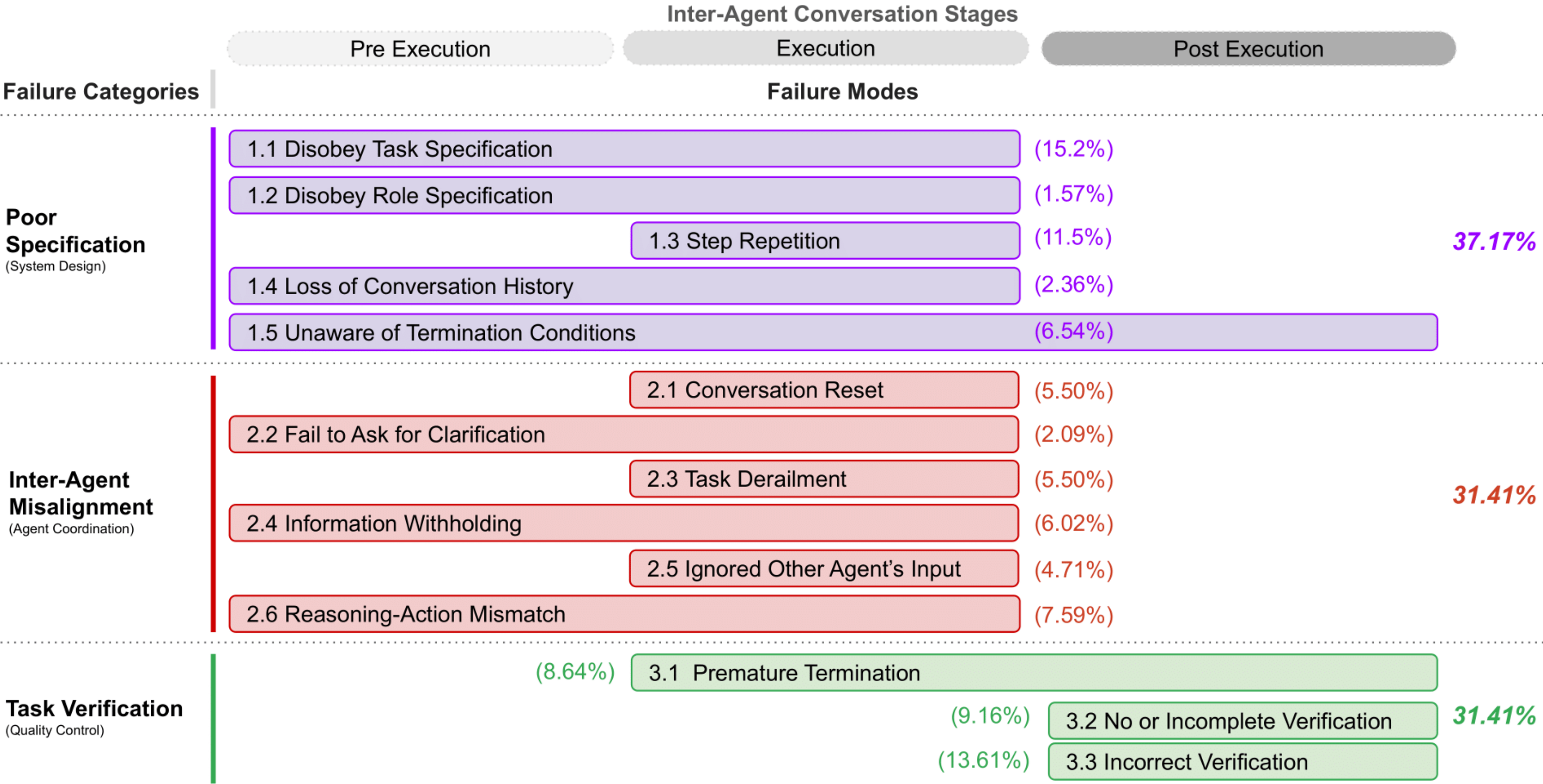
((b)) **Plan-Execute** Approach



# Multi-Agent System Taxonomy's Core Failure Categories

Three Main Failure Categories:

- **Specification Failures:** Misunderstood tasks, ambiguous goals, or unclear success metrics.
  - Example: Agent reports success, but output doesn't match user intent.
- **Inter-Agent Failures:** Poor coordination or lack of shared context.
  - Example: One agent updates a plan, but others act on outdated data.
- **Task Verification Failures:** Missing validation, poor execution tracking.
  - Example: Action taken, but no check if effect occurred.





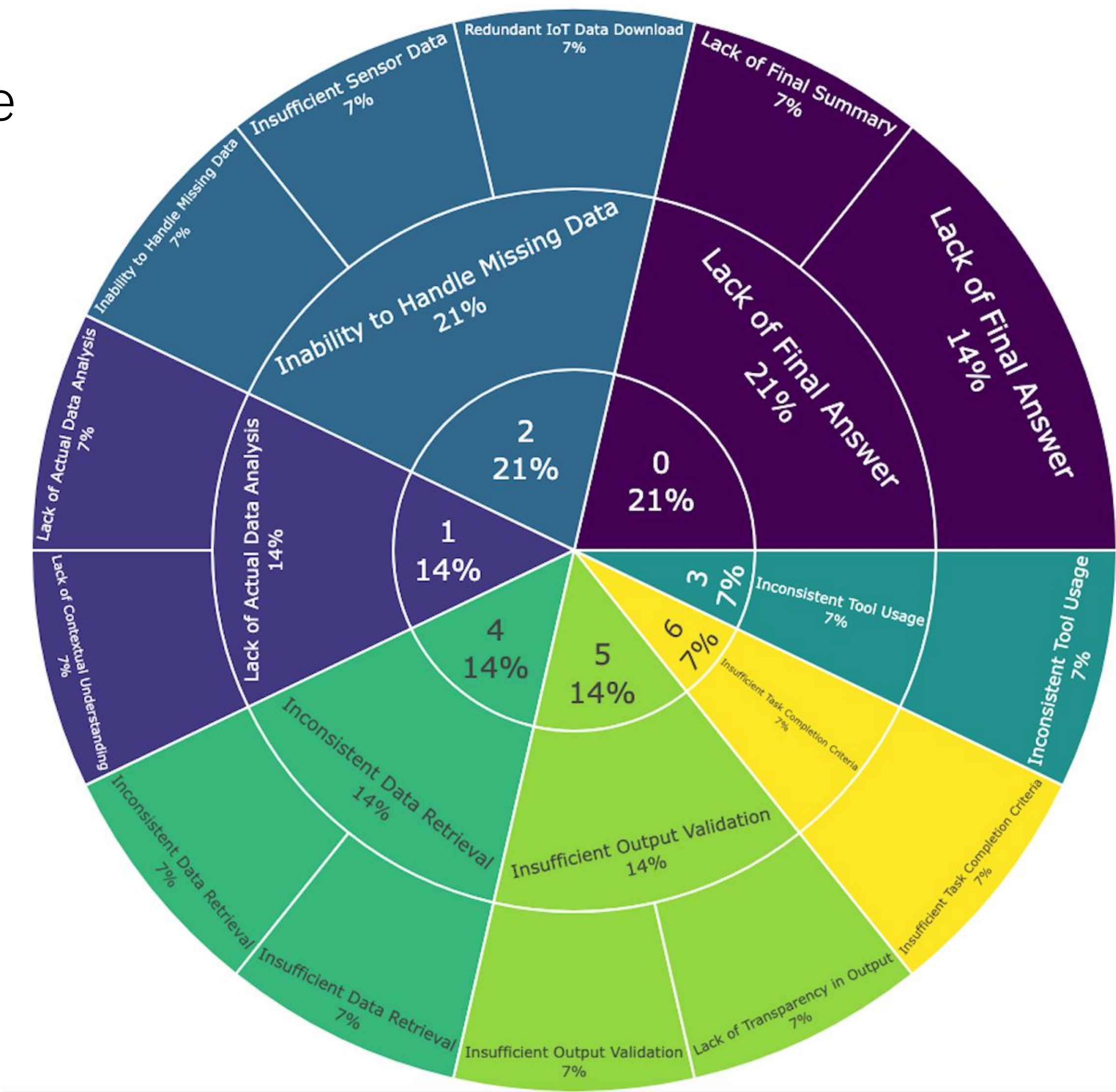
# Emergent Failure Mode Discovery

We have an automated way of trajectory introspection to discover novel, emerging failure modes automatically

Agent Coordination (Total 27.52%)	
Conversation Reset	Execution: 0.00%
Fail to Ask for Clarification	Execution: 10.22%
Task Derailment	Execution: 4.34%
Information Withholding	Execution: 2.22%
Ignored Agent's Input	Execution: 2.06%
Action Mismatch	Execution: 8.68%

We added a new method called “Self-Ask” to demonstrate the performance improvement

Model	enable_agent_ask=True	enable_agent_ask=False
gpt-4.1-2025-04-14	63%	65%
lama-4-maverick	66%	59%
llama-3-405b-instruct	61%	44%
mistral-large	58%	40%
llama-3-3-70b-instruct	35%	40%
granite-3-3-8b-instruct	32%	35%





# AssetOpsBench Extensive Research: FailureSensorIQ

- FailureSensorIQ introduces a **dataset** and **benchmark** that tests whether LLMs can reason about sensors, assets, and failure modes beyond data-driven correlations. It benchmarks *sensor-failure relationships*, which is the primary capability targeted by the Failure Mode Sensor Relation (FMSR) agent in AssetOpBench.
- FailureSensorIQ is accepted in NeurIPS 2025.

Datasets:

ibm-research

FailureSensorIQ

like

3

Follow

IBM Research

390

Tasks:

Question Answering

Modalities:

Text

Formats:

json

Languages:

English

Size:

1K - 10K

ArXiv:

Libraries:

Datasets

pandas

Croissant

+ 1

License:

apache-2.0

Dataset card

Data Studio

Files and versions

xet

Community

9

Dataset Viewer

Auto-converted to Parquet

API

Embed

Data Studio

Subset (2)

multi\_true\_multi\_choice\_qa · 5.63k rows

Split (1)

train · 5.63k rows

Search this dataset

subject	id	question	options	option_ids
string · classes	int64	string · classes	list · lengths	list · lengths
1 value	1	5.63k	327 values	5
failure_mode_sensor_analysis	1	For electric motor, if a...	[ "oil debris", ...	[ "A", "B", "C", "D", "E"...
failure_mode_sensor_analysis	2	For electric motor, if a...	[ "resistance", ...	[ "A", "B", "C", "D", "E"...
failure_mode_sensor_analysis	3	For electric motor, if a...	[ "coast down time", ...	[ "A", "B", "C", "D", "E"...
failure_mode_sensor_analysis	4	For electric motor, if a...	[ "partial discharge", ...	[ "A", "B", "C", "D", "E"...
failure_mode_sensor_analysis	5	For electric motor, if a...	[ "temperature", ...	[ "A", "B", "C", "D", "E"...

HuggingFace Dataset

IBM RESEARCH · BENCHMARK · V1

0

Share

FailureSensorIQ

A Multi-Choice QA (MCQA) dataset that explores the relationships between sensors and failure modes for 10 industrial assets.

Leaderboard

Discussion (0)

FailureSensorIQ is a novel Multi-Choice Question-Answering benchmarking system designed to assess the ability of Large Language Models to reason and understand complex, domain-specific scenarios in Industry 4.0. Unlike traditional QA benchmarks, our system focuses on multiple aspects of reasoning through failure modes, sensor data, and the relationships between them across various industrial assets

Results independently reproduced by Kaggle. Learn more

Last updated November 23, 2025

#	Model	Score	Consistency	F-Score	Elimination Accuracy
1	Gemini-3-Pro-Preview	69.1%	63.8%	69.1%	78.8%
2	O3-2025-04-16	67.6%	59.1%	67.4%	69.4%
3	Gpt-5-2025-08-07	67.2%	59.2%	67.3%	69.4%
4	Gemini-2.5-Pro	67.8%	57.5%	67.8%	68.8%
5	Gemini-2.5-Flash	65.5%	56.1%	65.8%	68.3%
6	Gpt-5-Mini-2025-08-07	65.3%	56.8%	65.5%	68.2%
7	O4-Mini-2025-04-16	64.8%	56.7%	65.8%	67.4%
8	Grok-4.1-Fast-Reasoning	64.6%	57.9%	64.9%	66.4%

Kaggle Benchmark

# Learn More

## Contact Information:

Dhaval Patel  
[pateldha@us.ibm.com](mailto:pateldha@us.ibm.com)

Nianjun Zhou  
[jzhou@us.ibm.com](mailto:jzhou@us.ibm.com)

Shuxin Lin  
[shuxin.lin@ibm.com](mailto:shuxin.lin@ibm.com)

[AssetOpsBench](#)  
[Github Repo](#)



[AssetOpsBench](#)  
[Arxiv Paper](#)



---

## Upcoming event announcement:

- Please come and Attend